

Fundamentals of Computer Programming (I) **—— C Programming Language**

Basic Data Types in C Language

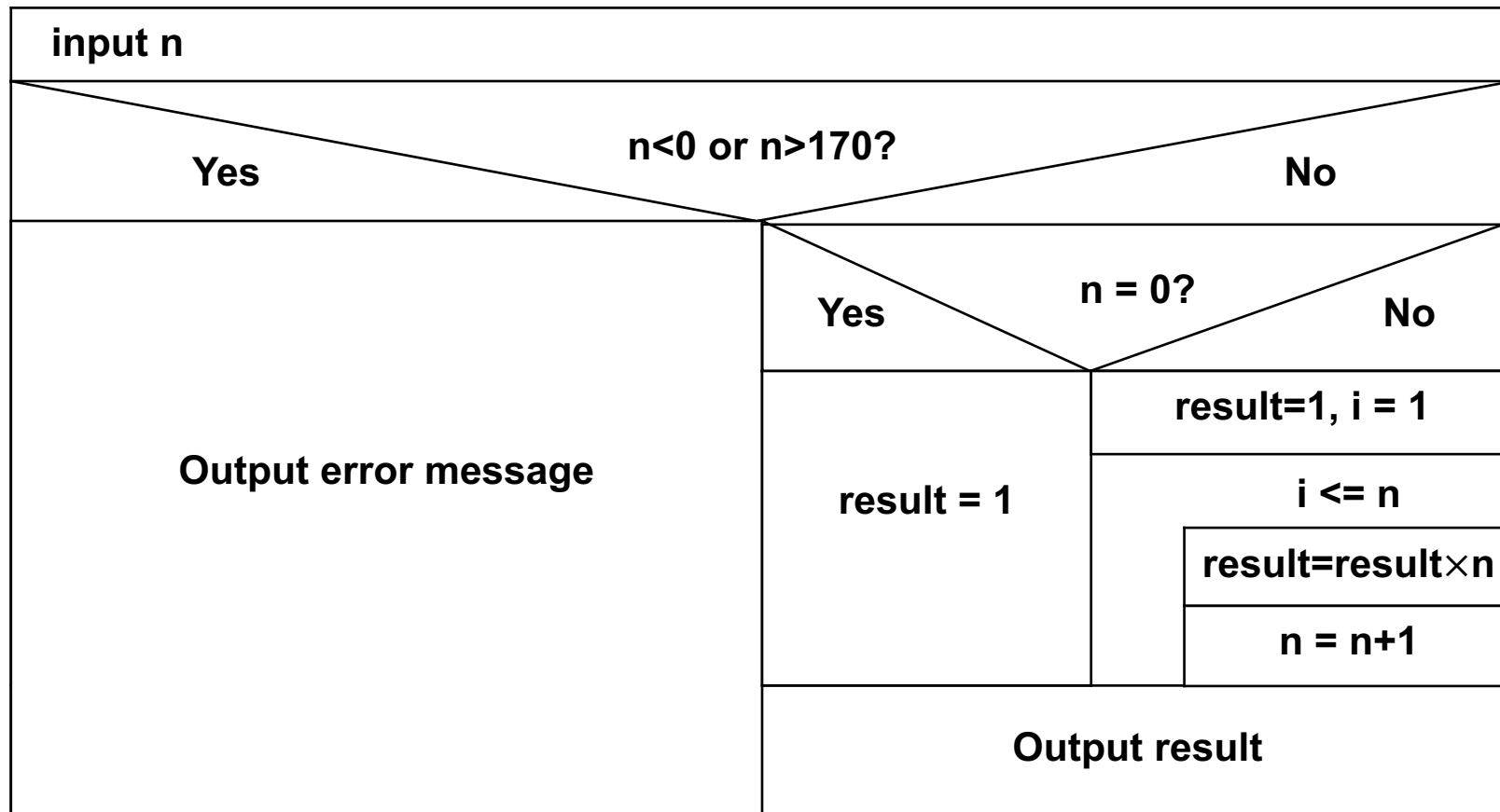
Xing Lin, Assistant Professor
Institute of Information Optoelectronics
Department of Electronic Engineering



Office: Rohm Building 5-208, Tel: 010-6279-2127, Email: lin-x@tsinghua.edu.cn

Pre-class Questions

- Write a C program to input a non-negative integer n and output its factorial result in the format: $n!=\text{result}$



Pre-class Questions

- Write a C program to input a non-negative integer n and output its factorial result in the format: $n!=\text{result}$

```
#include <stdio.h>
int main()
{
    int n;
    double result;
    printf("Please input a non-negative integer n ( $0 \leq n \leq 170$ ): ");
    scanf_s("%d", &n);
    if (n < 0 || n > 170) {
        printf("Error: Input must be a non-negative integer and not exceed! \n");
        return 1;
    }
    if (n == 0)
        result = 1; //  $0! = 1$ 
```

Pre-class Questions

- Write a C program to input a non-negative integer n and output its factorial result in the format: $n!=\text{result}$

```
else
{
    result = 1;

    for (int i = 1; i <= n; i++) {
        result *= i; // Iterative multiplication accumulation
    }
}
printf("%d! = %f\n", n, result);
return 0;
}
```

Course Syllabus

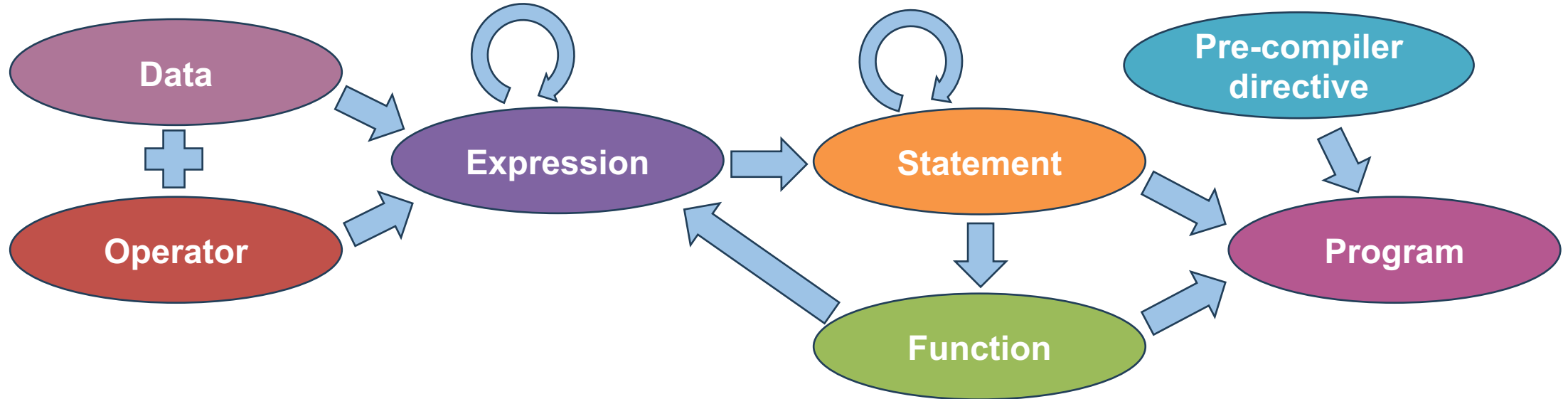
- Lecture 1: Course Introduction
- **Lecture 2: Basic Data Types in C Language**
- Lecture 3: Data Input and Output
- Lecture 4: C Expressions and Macro Definitions
- Lecture 5: Selection Structures
- Lecture 6: Compilation Preprocessing
- Lecture 7: Loop Structure
- Lecture 8: Module Design
- Lecture 9: Arrays
- Lecture 10: Pointers (I)
- Lecture 11: Pointers (II)
- Lecture 12: Structures and Unions (I)
- Lecture 13: Structures and Unions (II)
- Lecture 14: Files
- Lecture 15: Bitwise Operation
- Course Summary and Review

Chapter Syllabus

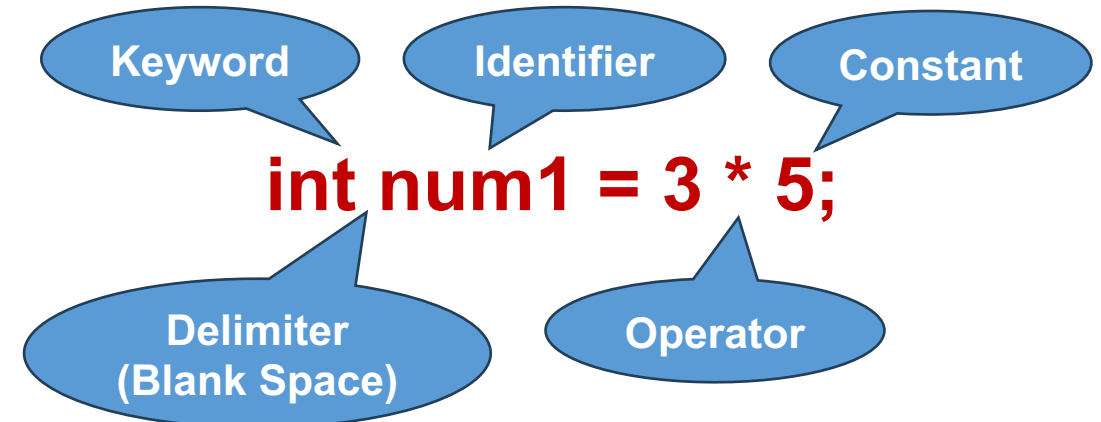
- **2.1 Representation of Data in Computers**
 - 2.1.1 Computer Number Systems**
 - 2.1.2 Representation of Numbers in Computers**
- **2.2 Constants and Variables**
- **2.3 Basic Data Type Constants**
 - 2.3.1 Integer Constants**
 - 2.3.2 Floating-point Constants**
 - 2.3.3 Character Constants**
- **2.4 Definition of Basic Data Type Variables**
 - 2.4.1 Definition of Integer Variables**
 - 2.4.2 Definition of Floating-point Variables**
 - 2.4.3 Definition of Character Constants**

Why start from "data"?

- The composition and pipeline of C/C++ programming



- Statement is the basic unit of C/C++
- Statement = "Expression" + ";"
- Expression = **Data** + Operator



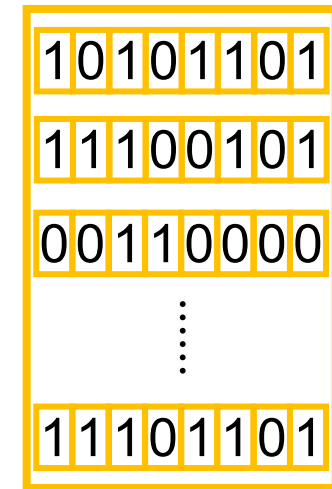
Computer number system

- In the decimal system: the **positional weight** is a power of the base 10.
- In the binary system (invented by Leibniz, inspired by “I Ching”): there is **only two numerical symbols: 0 and 1**. Its characteristic is that it “adds one when encountering two”. Similar to decimal numbers, in binary numbers, each digit (0 or 1) has a different value depending on its position, and the **positional weight** is a power of the base 2.
- Example: $(97)_{10} = 9 \times 10^1 + 7 \times 10^0$, $(1100001)_2 = 1 \times 2^6 + 1 \times 2^5 + 0 + 1 \times 2^0$
- **Binary is easy to represent (only two states)**: (1) Light bulb: on, off; (2) Diode: conducting, non-conducting; (3) Voltage level: high, low; (4) Pit: existing, non-existing (VCD, DVD discs); (5) Logic: true, false.

- **There are 10 kinds of people in the world, one knows binary, and one doesn't**

Computer number system

- In a computer, data is stored in memory in "binary" form **due to its efficiency, reliability, and simplicity.**
 - The **smallest storage unit in memory is a bit**, representing 0 or 1.
 - The byte is the most basic unit for memory allocation and usage, usually **each byte consisting of 8 bits.**
-
- **Data representation in C/C++ languages is based on bytes.**
 - **Each byte contains 8 bits** and can represent 256 $[0, 2^8-1]$ different values.
 - The space occupied by different data types is always an **integer multiple of bytes.**



Computer memory structure

Computer number system

Conversion Relation	Conversion Relation
Minimum unit: {0,1}	1ZB (Zettabyte) = 1024 EB $\approx 10^{21}$ B
1 Byte = 8 bit	1YB (Yottabyte) = 1024 ZB $\approx 10^{24}$ B
1KB (Kilobyte) = 1024 B $\approx 10^3$ (2^{10}) B	1BB (Brontobyte) = 1024 YB $\approx 10^{27}$ B
1MB (Megabyte) = 1024 KB $\approx 10^6$ B	
1GB (Gigabyte) = 1024 MB $\approx 10^9$ B	
1TB (Trillionbyte) = 1024 GB $\approx 10^{12}$ B	
1PB (Petabyte) = 1024 TB $\approx 10^{15}$ B	
1EB (Exabyte) = 1024 PB $\approx 10^{18}$ B	

How big is 1 byte?

- Represent numbers: 0~255
- Save a character: English letters, numbers, symbols
 - ASCII code
- Two bytes to save a Chinese character
 - GB2312, 6763 word
 - GB18030, 27533 word
 - BIG5, 13000 word

0	1	0	0	1	0	0	1
0	0	1	0	0	0	0	0

Computer number system

Numeral System	Elements	Rule	Example
Decimal System	{0, 1, 2,..., 9}	Carry over at ten	9+1=10
Sexagesimal System	{0, 1, 2,..., 59}	Carry over at sixty	11:59:59 AM
Binary System	{0, 1}	Carry over at two	1+1=10
Octal System	{0, 1, 2,..., 7}	Carry over at eight	7+1=10
Hexadecimal System	{0,1,...,9,A,B,C,D,E,F}	Carry over at sixteen	F+1=10

Convert decimal integers to binary integers

● Example 2.1: Converting the decimal number $(97)_{10}$ to a binary number

2|97

2|48 the remainder is 1, which is $a_0 = 1$

2|24 the remainder is 0, which is $a_1 = 0$

2|12 the remainder is 0, which is $a_2 = 0$

2|6 the remainder is 0, which is $a_3 = 0$

2|3 the remainder is 0, which is $a_4 = 0$

2|1 the remainder is 1, which is $a_5 = 1$

0 the remainder is 1, which is $a_6 = 1$;

Quotient is 0, end

Modulus 2 method: (1) Divide the decimal number by 2 to get a quotient and a remainder; (2) then divide the quotient by 2 again to get another quotient and remainder; (3) continue this process until the quotient equals zero. (4) Each remainder obtained (which must be 0 or 1) corresponds to each digit of the binary number.

Note: The first remainder obtained is the lowest bit of the binary number, and the last remainder obtained is the highest bit of the binary number.

The final result is:

$$(97)_{10} = (a_6 a_5 a_4 a_3 a_2 a_1 a_0)_2 = (1100001)_2$$

Convert binary integers to decimal integers

- **Example 2.1b: Converts a binary number $(1100001)_2$ to a decimal number**

(1	1	0	0	0	0	1)	₂
----	---	---	---	---	---	----	--------------

2^6	2^5	2^4	2^3	2^2	2^1	2^0
-------	-------	-------	-------	-------	-------	-------

64	32	16	8	4	2	1
----	----	----	---	---	---	---

$$(1100001)_2 = 64 + 32 + 1 = (97)_{10}$$

- Multiply the n-th bit of the binary number by the positional weight 2^n , and sum up the value of each bit.
- $(a_n a_{n-1} \dots a_2 a_1 a_0)_2 = (a_n \times 2^n + a_{n-1} \times 2^{n-1} + \dots + a_2 \times 2^2 + a_1 \times 2^1 + a_0 \times 2^0)_{10}$

Convert decimal fraction to binary fraction

- **Example 2-2: The procedure for converting the decimal fraction 0.6875 to a binary fraction is as follows:**

0.6875

x 2

1.3750 The integer part is 1, $a_{-1} = 1$

0.3750 The rest of the fraction part

x 2

0.7500 The integer part is 0, $a_{-2} = 0$

0.7500 The rest of the fraction part

x 2

1.5000 The integer part is 1, $a_{-3} = 1$

0.5000 The rest of the fraction part

x 2

1.0000 The integer part is 1, $a_{-4} = 1$

0.0000 The rest of the fraction part is 0

The final result is $(0.6875)_{10} = (0.a_{-1} a_{-2} a_{-3} a_{-4})_2 = (0.1011)_2$

“Double and Round” method: (1)

Multiply the decimal fraction by 2 to obtain an integer part and a fractional part. **(2)** Then multiply the fractional part by 2 again to obtain another integer part and a fractional part. **(3)** Continue this process until the remaining fractional part is 0 or meets the precision requirement. **(4)** Finally, arrange the integer parts (necessarily 0 or 1) obtained each time from left to right to get the corresponding binary fraction.

Convert decimal fraction to binary fraction

- **A decimal fraction may not convert precisely to binary.** If so, the conversion can end at a specific decimal **based on precision needs**. For instance, **0.32 in decimal can't be accurately converted to binary.** The process is:

0.32	
x 2	
0.64	The integer part is 0, $a_1 = 0$
x 2	
1.28	The integer part is 1, $a_2 = 1$
0.28	The rest of the fraction part
x 2	
0.56	The integer part is 0, $a_3 = 0$
x 2	
1.12	The integer part is 1, $a_4 = 1$
0.12	The rest of the fraction part
x 2	
0.24	The integer part is 0, $a_5 = 0$

0.24	
x 2	
0.48	The integer part is 0, $a_6 = 0$
x 2	
0.96	The integer part is 0, $a_7 = 0$
x 2	
1.92	The integer part is 1, $a_8 = 1$
0.92	The rest of the fraction part
x 2	
1.84	The integer part is 1, $a_9 = 1$
0.84	The rest of the fraction part
x 2	
1.68	The integer part is 1, $a_{10} = 1$
.....	

Convert decimal fraction to binary fraction

- The decimal 0.32 can't be converted precisely to a binary fraction due to an endless conversion process. **Stopping at a specific point after the binary fraction yields an approximate value.**
- For example, at the **4-th** position, we get: $(0.32)_{10} \approx (0.0101)_2$. In fact, the corresponding decimal fraction of this binary fraction is: $(0.0101)_2 = (0.3125)_{10}$
- If we require to take it to the **8-th** digit after the binary decimal point, we can get: $(0.32)_{10} \approx (0.01010001)_2$. In fact, the corresponding decimal fraction of this binary fraction is: $(0.01010001)_2 = (0.31640625)_{10}$

Convert binary fraction to decimal fraction

- **Example: Converts the binary fraction $(0.1011)_2$ to a decimal decimal**

$$\begin{array}{ccccccc}(0. & 1 & & 0 & & 1 & & 1 &)_2 \\ & 2^{-1} & & 2^{-2} & & 2^{-3} & & 2^{-4} & \\ & 0.5 & & 0.25 & & 0.125 & & 0.0625 & \end{array}$$

$$\begin{aligned}(0.1011)_2 &= 0.5 + 0.125 + 0.0625 \\ &= 0.625 + 0.0625 \\ &= (0.6875)_{10}\end{aligned}$$

- **Example: Converts the binary decimal $(0.01010001)_2$ to a decimal decimal**

$$\begin{array}{cccccccc}(0. & 0 & & 1 & & 0 & & 1 & & 0 & & 0 & & 0 & & 1 &)_2 \\ & 2^{-1} & & 2^{-2} & & 2^{-3} & & 2^{-4} & & 2^{-5} & & 2^{-6} & & 2^{-7} & & 2^{-8} & \\ & 0.5 & & 0.25 & & 0.125 & & 0.0625 & & \dots & & 0.00390625 & & & & & \end{array}$$

$$\begin{aligned}(0.01010001)_2 &= 0.25 + 0.0625 + 0.00390625 \\ &= 0.3125 + 0.00390625 \\ &= (0.31640625)_{10}\end{aligned}$$

- **Multiply the n-th place of the binary decimal by the bit weight 2^{-n} and sum the values of each bit.**

$$(0.a_{-1} a_{-2} \dots a_{-n})_2 = (a_{-1} * 2^{-1} + a_{-2} * 2^{-2} + \dots + a_{-n} * 2^{-n})_{10}$$

Conversion of general decimal to binary numbers

- To convert a decimal number with both integer and fraction parts into a binary number, we can **convert its integer and fraction parts separately, and then combine them.**
- For example:

$$(97)_{10} = (1100001)_2$$

$$(0.6875)_{10} = (0.1011)_2$$

$$\text{Thus, we have: } (97.6875)_{10} = (1100001.1011)_2$$

Hexadecimal system

- The hexadecimal value: **add 1 every 16, with A to F representing 10 to 15**
- Convert decimal integers to hexadecimal integers with **modulus 16 method**
- **Example 2-3: converts the decimal number 986 to hexadecimal**

16|9 8 6

16|6 1 the remainder is 10, $a_0 = A$

16|3 the remainder is 13, $a_1 = D$

0 the remainder is 3, $a_2 = 3$; quotient is 0, end

The final result is:
 $(986)_{10} = (a_2 a_1 a_0)_{16} = (3DA)_{16}$

- **Convert hexadecimal integers to decimal integers:** Multiply the n-th digit of a hexadecimal integer by its positional weight 16^n , and then sum up the value of each digit.
- **Example 2-3B: converts a hexadecimal number $(3DA)_{16}$ to a decimal number**

$$(3DA)_{16} = (3 \cdot 16^2 + 13 \cdot 16^1 + 10 \cdot 16^0)_{10} = (986)_{10}$$

Convert decimal fraction to hexadecimal fraction

- **Example 2-4: Converts the decimal fraction 0.84375 to a hexadecimal fraction**

Multiply by 16 and round method

$$\begin{array}{r} 0.84375 \\ \times \quad 16 \\ \hline \end{array}$$

13.50000

0.50000

$$\begin{array}{r} \times \quad 16 \\ \hline \end{array}$$

8.00000

0.00000

The integer part is 13, $a_{-1} = D$

The rest of the fraction part

The integer part is 8, $a_{-2} = 8$

The rest of the fraction part is 0, end

The final result is:
 $(0.84375)_{10} = (0.D8)_{16}$

Convert decimal fraction to hexadecimal fraction

- **Similarly, decimal fraction may not convert accurately to hexadecimal.** Depending on accuracy needs, conversion can be limited to a specific decimal place. **For instance, 0.32 in decimal cannot be precisely converted to hexadecimal.** The process is:

0. 3 2

x 1 6

5. 1 2

0. 1 2

x 1 6

1. 9 2

0. 9 2

x 1 6

.....

The integer part is 5, $a_{-1} = 5$

The rest is the fraction part

The integer part is 1, $a_{-2} = 1$

The rest is the fraction part

Convert decimal fraction to hexadecimal fraction

- The decimal 0.32 can't be converted precisely to a hexadecimal fraction due to an endless conversion process. **Stopping at a specific point after the hexadecimal fraction yields an approximate value.**
- For example, at the **1-th** position, we get: $(0.32)_{10} \approx (0.5)_{16}$. In fact, the corresponding decimal fraction of this binary fraction is: $(0.5)_{16} = (0.3125)_{10}$
- If we require to take it to the **2-th** digit after the binary decimal point, we can get: $(0.32)_{10} \approx (0.51)_{16}$. In fact, the corresponding decimal fraction of this binary fraction is: $(0.51)_{16} = (0.31640625)_{10}$

Convert hexadecimal fraction to decimal fraction

- Multiply the n-th digit of the hexadecimal decimal **by the positional weight of 16^{-n}** , summing the values of each digit
- **Example 2-4B: Converts the hexadecimal fraction $(0.D8)_{16}$ to a decimal**

$$(0. \quad D \quad 8)_{16}$$

$$16^{-1} \quad 16^{-2}$$

$$0.0625 \quad 0.00390625$$

Final result:

$$(0.D8)_{16} = (13 \cdot 16^{-1} + 8 \cdot 16^{-2})_{10}$$

$$= (13 \cdot 0.0625 + 8 \cdot 0.00390625)_{10}$$

$$= (0.84375)_{10}$$

Conversion of general decimal to hexadecimal

- To convert a decimal number with both integer and decimal parts into a hexadecimal number, we can **convert its integer and fraction parts separately, and then combine them.**
- **Example 2-5: converts the decimal 986.84375 to hexadecimal number**

- Convert the **integer part** first: $(986)_{10} = (3DA)_{16}$
- Convert the **fractional part**: $(0.84375)_{10} = (0.D8)_{16}$
- The **final result** is: $(986.84375)_{10} = (3DA.D8)_{16}$

- Similar representation and conversion method hold for **Octal system**

Conversion between computer number systems

	Decimal	Binary	Octal	Hexadecimal
Base number	10	2	8	16
Positional weight	10^K	2^K	8^K	16^K
Numeric symbol	0~9	0, 1	0~7	0~9 and A~F

Conversion between computer number systems

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Convert hexadecimal and octal numbers to binary

- **Example 2-8: Hexadecimal number $(2BD.C)_{16}$ converted to binary number**

2	B	D	.	C
↓	↓	↓	↓	↓
0010	1011	1101	.	1100

Each hexadecimal number is replaced by the corresponding four-bit binary number

$$(2BD.C)_{16} = (1010111101.11)_2$$

- **Example 2-9: Octal number $(315.27)_8$ converts to binary number**

3	1	5	.	2	7
↓	↓	↓	↓	↓	↓
011	001	101	.	010	111

Each octal digit is replaced by the corresponding three binary digits

$$(315.27)_8 = (11001101.010111)_2$$

Convert binary numbers to hexadecimal and octal

- **Example 2-10: Convert the binary number $(1101001101.01)_2$ to hexadecimal**

$$\begin{array}{cccccc} 11 & 0100 & 1101 & . & 0100 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 3 & 4 & D & . & 4 \end{array}$$
$$(1101001101.01)_2 = (34D.4)_{16}$$

Starting from the fraction point, every four bits forward constitute a hexadecimal number; every four bits backward also constitute a hexadecimal number. When the last group has less than four bits, zeros should be added to make up four bits.

- **Example 2-11: Convert the binary number $(1101001101.01)_2$ to octal**

$$\begin{array}{cccccc} 1 & 101 & 001 & 101 & . & 010 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 5 & 1 & 5 & . & 2 \end{array}$$
$$(1101001101.01)_2 = (1515.2)_8$$

Starting from the decimal point, every three bits forward form a group to constitute an octal digit; similarly, every three bits backward form a group to constitute an octal digit. When the last group does not have three bits, zeros should be added at the end to make up three bits.

Chapter Syllabus

- **2.1 Representation of Data in Computers**
 - 2.1.1 Computer Number Systems
 - 2.1.2 Representation of Numbers in Computers**
- **2.2 Constants and Variables**
- **2.3 Basic Data Type Constants**
 - 2.3.1 Integer Constants
 - 2.3.2 Floating-point Constants
 - 2.3.3 Character Constants
- **2.4 Definition of Basic Data Type Variables**
 - 2.4.1 Definition of Integer Variables
 - 2.4.2 Definition of Floating-point Variables
 - 2.4.3 Definition of Character Constants

Representation of positive and negative numbers

- In computers, a number's sign is represented by **a binary bit**
- Usually use the **highest bit of the binary number**
- **"0" and "1"** sign bits indicate the **positive and negative numbers**, respectively

8-bit

The maximum unsigned number is
255 (2^8-1)

The signed integer ranges from
-128~127 (2^7-1)

16-bit

The maximum unsigned number is
65535 ($2^{16}-1$)

The signed integer ranges from
-32768~32767 ($2^{15}-1$)

32-bit

The maximum unsigned number is
4294967295 ($2^{32}-1$)

The signed integer ranges from
-2147483648~2147483647 ($2^{31}-1$)

64-bit

The maximum unsigned number is
18446744073709551615 ($2^{64}-1$)

The signed integer ranges from
**-9223372036854775808 ~
9223372036854775807
($2^{63}-1$)**

Representation of positive and negative numbers

- For example, +50 and -50 in decimal can be represented in **8-bit, 16-bit, and 32-bit binary** with the corresponding hexadecimal conversion:

$$(+50)_{10} = (00110010)_2 = (32)_{16}$$

$$(-50)_{10} = (10110010)_2 = (B2)_{16}$$

$$(+50)_{10} = (000000000110010)_2 = (0032)_{16}$$

$$(-50)_{10} = (10000000000110010)_2 = (8032)_{16}$$

[illegible]

[illegible]

- **Similarly, +513 and -513 in decimal can be represented in 16-bit binary, with the corresponding hexadecimal conversion:**

$$(+513)_{10} = (0000001000000001)_2 = (0201)_{16}$$

$$(-513)_{10} = (1000001000000001)_2 = (8201)_{16}$$

Fixed-point number

- A fixed-point number has a fixed decimal position, which is often used in computers to represent integers (fixed-point integers) and pure decimals (fixed-point fractions).

(1) Fixed-point integer

- Highest bit in a fixed-point integer is the sign bit, indicating the number's sign
- Decimal point is implicitly after the rightmost bit, without occupying a separate bit
- Bits right of the sign bit represent the integer, i.e., the fixed-point integer value

(2) Fixed-point fraction

- Highest bit in a fixed-point decimal is the sign bit, indicating the number's sign
- The decimal point follows the sign bit implicitly, without occupying a separate bit
- Bits right of the sign bit represent the fraction, i.e., the fixed-point fraction value

True form, ones' and two's complement, offset binary

(1) True form

- **The true form is a binary fixed-point representation** where the sign bit is at the highest position, and the numerical part follows standard binary format. For example, the 8-bit binary true form of $(+50)_{10}$, $(-50)_{10}$, $(+33)_{10}$, $(-33)_{10}$ are

$(00110010)_{TF}$, $(10110010)_{TF}$, $(00100001)_{TF}$, $(10100001)_{TF}$

- With more binary bits in true form, a wider range of numbers can be represented. Using **n bits for a fixed-point integer** in true form yields a range of

$$-2^{n-1}+1 \sim 2^{n-1}-1$$

True form, ones' and two's complement, offset binary

(2) One's complement

- The one's complement of a **positive number** equals its original code, while a **negative number**'s one's complement **inverts all non-sign bits**. For example, the 8-bit binary one's complement of $(+50)_{10}$, $(-50)_{10}$, $(+33)_{10}$, $(-33)_{10}$ are

$(00110010)_{OC}$, $(11001101)_{OC}$, $(00100001)_{OC}$, $(11011110)_{OC}$

- $(00000000)_{OC}$ represents the decimal number: 0
- $(11111111)_{OC}$ represents the decimal number: -0

True form, ones' and two's complement, offset binary

(3) Two's complement

- **Positive numbers** have the same two's complement with respect to the original code. **For negatives**, the two's complement is the one's complement **plus 1 at the rightmost bit**.
- **The two's complement of a two's complement is the original code.**
- **With n-bit binary for fixed-point integers, the range is: $-2^{n-1}+1 \sim 2^{n-1}-1$.**

[Example 2-13] Use two's complement to calculate: $(33)_{10} - (50)_{10}$ and $(33)_{10} + (50)_{10}$

- $(33)_{10} - (50)_{10} = (-50)_{10} + (+33)_{10} = (11001110)_{TC} + (00100001)_{TC} = (11101111)_{TC} = (10010001)_{TF} = (-17)_{10}$
- $(33)_{10} + (50)_{10} = (+50)_{10} + (+33)_{10} = (00110010)_{TC} + (00100001)_{TC} = (01010011)_{TC} = (10010001)_{TF} = (83)_{10}$

Tip: Computers use two's complement codes for simplified arithmetic without considering the sign bit during the calculation.

True form, ones' and two's complement, offset binary

(4) Offset binary

- The offset binary **inverts the sign bit of a number's two's complement**. In offset representation, 1 is positive, 0 is negative, and the numeric part remains the same as in two's complement.
- **Fixed-point arithmetic requires inverting the sign bit of the result.**
- With n-bit binary for fixed-point integers, the range is: **$-2^{n-1}+1 \sim 2^{n-1}-1$** .

Examples:

$$(33)_{10} = (00100001)_{TC} = (10100001)_{OB}$$

$$(-50)_{10} = (11001110)_{TC} = (01001110)_{OB}$$

$$(0)_{10} = (00000000)_{TC} = (10000000)_{OB}$$

$$(-128)_{10} = (10000000)_{TC} = (00000000)_{OB}$$

$$(127)_{10} = (01111111)_{TC} = (11111111)_{OB}$$

True form, ones' and two's complement, offset binary

(4) Offset binary

- Examples:

$$\begin{aligned}(-50)_{10} + (33)_{10} &= (01001110)_{OB} + (10100001)_{OB} \\&= (01101111)_{OB} && \text{Invert the sign bit of the result} \\&= (10010001)_{TF} && \text{Invert all bits and add 1 to last bit} \\&= (-17)_{10}\end{aligned}$$

In the four representations of binary fixed-point numbers, the true form is intuitive but unusable for calculations, while two's complement and offset binary are operable. The ones' complement aids conversion.

Floating point numbers

- A decimal number R with integer and fraction parts can be expressed as:

$$R = Q \times 10^n,$$

where Q is a pure fraction and n is an integer.

- Similarly, a binary number P with integer and fraction parts can be expressed as:

$$P = S \times 2^N,$$

where the mantissa S is a binary fixed-point fraction; and the exponent N is a binary fixed-point integer, representing the decimal point's actual position.

- Binary floating-point numbers are stored with binary bits and structured as mantissa and exponent:

Mantissa S (Fixed-point fraction with TF or TC code)	Exponent N (Fixed-point integer with TF or OB code)
---	--



The most significant bit (MSB) of binary



The least significant bit (LSB) of binary

Floating point numbers

[Example 2-14] Represent -254.75 as a **16-bit binary fixed-point fraction TC code** and an **8-bit binary fixed-point integer TC code**.

(1) First convert $(-254.75)_{10}$ to a binary number: $(-254.75)_{10} = (-11111110.11)_2 = (-0.1111111011)_2 \times 2^8$

(2) Then convert the mantissa to a 16-bit binary fixed-point fraction:

$$S = (-0.1111111011)_2 = (\overset{\uparrow}{1} 1 1 1 1 1 1 0 1 1 0 0 0 0 0)_2$$

Point position

(3) Its one's complement code is: $S = (1000000010011111)_{OC}$

(4) Its two's complement code is: $S = (1000000010100000)_{TC} = (80A0)_{16}$

(5) Convert exponent 8 to binary as well, $(+8)_{10} = (+1000)_2$

(6) Convert to an 8-bit binary fixed-point integer: $N = (+1000)_2 = (00001000)_2$

Point position

(7) Its two's complement code is: $N = (00001000)_{TC} = (08)_{16}$

The final representation of $(-254.75)_{10}$ in hexadecimal is $(80A008)_{16}$

Chapter Syllabus

- **2.1 Representation of Data in Computers**
 - 2.1.1 Computer Number Systems
 - 2.1.2 Representation of Numbers in Computers
- **2.2 Constants and Variables**
- **2.3 Basic Data Type Constants**
 - 2.3.1 Integer Constants
 - 2.3.2 Floating-point Constants
 - 2.3.3 Character Constants
- **2.4 Definition of Basic Data Type Variables**
 - 2.4.1 Definition of Integer Variables
 - 2.4.2 Definition of Floating-point Variables
 - 2.4.3 Definition of Character Constants

Constants and variables

- **Constants:** unchanging values during program execution
- **Variables:** values that can change during program execution

C language's basic data types

Command: `sizeof(keyword)`

Data Type	Keyword	Length Description	Value Range
Integer data	(unsigned) short int	2B (un)signed integer	$[0, 2^{16}-1]$ or $[-2^{15}, 2^{15}-1]$
	(unsigned) int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$
	(unsigned) long int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$
	(unsigned) long long	8B (un)signed long integer	$[0, 2^{64}-1]$ or $[-2^{63}, 2^{63}-1]$
Floating-point data or real number data	float	4B floating-point number	$\sim(-10^{38} \sim 10^{38})$
	double	8B floating-point number	$\sim(-10^{308} \sim 10^{308})$
	long double	8B floating-point number	$\sim(-10^{308} \sim 10^{308})$

Constants and variables

- **Constants:** unchanging values during program execution
- **Variables:** values that can change during program execution

C language's basic data types

Command: `sizeof(keyword)`

Data Type	Keyword	Length Description	Value Range
Character data	<code>char</code>	1B signed character	$[-2^7, 2^7-1]$
	<code>unsigned char</code>	1B unsigned character	$[0, 2^8-1]$
	<code>bool</code>	1B unsigned character	$\{0,1\}$
Void data	<code>void</code>	0B	—

- Besides the four basic data types, C also has composite types, including: enumerations, structures, pointers, etc.

Variable name

- **Variable name:** variables have unique names and must be defined before use
- **Defining a variable:** allocating storage based on its data type with the name representing this space for data access
- **Variable naming conventions:** start with a letter or underscore, followed by letters, numbers, or underscores, and are case-sensitive. "Day" and "day" are considered different variable names
- **Naming should be meaningful**, such as using the Hungarian notation
- **The number of characters in variable names** depends on the compiler system

Chapter Syllabus

- **2.1 Representation of Data in Computers**
 - 2.1.1 Computer Number Systems
 - 2.1.2 Representation of Numbers in Computers
- **2.2 Constants and Variables**
- **2.3 Basic Data Type Constants**
 - 2.3.1 Integer Constants
 - 2.3.2 Floating-point Constants
 - 2.3.3 Character Constants
- **2.4 Definition of Basic Data Type Variables**
 - 2.4.1 Definition of Integer Variables
 - 2.4.2 Definition of Floating-point Variables
 - 2.4.3 Definition of Character Constants

Integer constants

Data Type	Keyword	Length Description	Value Range
Integer data	(unsigned) short int	2B (un)signed integer	$[0, 2^{16}-1]$ or $[-2^{15}, 2^{15}-1]$
	(unsigned) int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$
	(unsigned) long int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$
	(unsigned) long long	8B (un)signed long integer	$[0, 2^{64}-1]$ or $[-2^{63}, 2^{63}-1]$

Representation of integer constants

(1) Decimal: Use numerals 0-9, +, and -

(2) Hexadecimal: Start with 0x/0X, use 0-9 and A-F/a-f

(3) Octal: Start with 0, use numerals 0-7

Integer constants

Rules in decimal notation for integer constants:

- "+" can be omitted for positives. E.g., +123 and 123 are equivalent.
- For long integers, add L or l.
- For unsigned, append U or u.
- For 64-bit, use i64, LL, or ll.
- E.g., 123u (unsigned int, 4 bytes), 123L (long int, 4 bytes), 123LL/123i64 (long long int, 8 bytes) have the same value but different storage sizes.

Floating-point (real number) constants

Rules in representing floating-point constants with C language:

- (1) Decimal notation: uses "+", "-", digits 0-9, and ".".
- (2) Scientific notation: adds "e" or "E" to decimal elements. The "e"/"E" precedes an integer. A number must precede "e", and no fractions after "e".

The storage of floating-point constants (IEEE 754 standard format):

- Real number constant: $P = S \times 2^N$, N is a binary fixed-point integer, denoting exponent; and S is a binary fixed-point fraction, denoting mantissa.

$$N = N' + 2$$

$$S = (-1)^{\text{sign}}(0.1S')$$

double 8 Byte	63	62	52	51	0
	Sign	Offset code of exponent (N') (11 bits)		True form code of mantissa (S') (52 bits)	
float 4 Byte	31	30	23	22	0
	Sign	Offset code of exponent (N') (8 bits)		True form code of mantissa (S') (23 bits)	

Floating-point (real number) constants

- **Verification: Print out the hexadecimal value of the internal representation of double and float type variables:**

```
#include <stdio.h>
int main() {
    double x = 97.6875;
    float y = 97.6875f;
    unsigned char* p;

    p = (unsigned char*)&x; // p points to the first byte of the 8-byte double precision number
    printf("%02X %02X %02X %02X %02X %02X %02X %02X\n",
        *p, *(p + 1), *(p + 2), *(p + 3), *(p + 4), *(p + 5), *(p + 6), *(p + 7));

    p = (unsigned char*)&y; // p points to the first byte of the 4-byte single precision number
    printf("%02X %02X %02X %02X\n", *p, *(p + 1), *(p + 2), *(p + 3));

    return 0;
} /* Note: %02X outputs the value stored in the pointed byte in two-digit hexadecimal format */
```

- The result on win10 VS2022:
00 00 00 00 00 6C 58 40
00 60 C3 42

- **Note: Since Intel CPU x64 uses little-endian byte order, the 2/4/8 bytes of a variable are stored in reverse order. Please learn about big-endian and little-endian on your own.**

Floating-point (real number) constants

- **Example 2-15:** The function of the following C program is to accumulate 10 real numbers of 0.1 and then output the accumulated result.

```
#include <stdio.h>
int main() {
    int k;           /* define integer variable k */
    double x, y, z;  /* define double precision floating-point variables */
    x = 0.0;
    y = 0.1;
    z = 1.0;         /* assign real number 1.0 to variable z */

    for (k = 0; k < 10; k++) {
        x = x + y;    /* accumulate 10 times 0.1 to variable x */
    }

    printf("x= %20.17f\n", x); /* output the value of variable x */
    printf("y= %20.17f\n", y); /* output the value of variable y */
    printf("z= %20.17f\n", z); /* output the value of variable z */
    return 0;
}
```

- After running this program, the output results are as follows:

z = 1.000000000000000000

x = 0.999999999999999989

There is an **accumulated error**.

- Due to limited storage space, a real constant usually can't be perfectly represented as a finite binary number, leading to **rounding errors**.

Character constants

- **Escape character**

'\n'	newline (line feed)
'\r'	carriage return (without newline)
'\b'	backspace
'\t'	tab (horizontal tab)
'\''	single quote (apostrophe)
'\"'	double quote (quotation mark)
'\ddd'	ASCII character represented by a 1 to 3 digit octal number
'\xhh'	ASCII character represented by a 1 to 2 digit hexadecimal number
'\f'	form feed (page break)
'\\'	backslash character
'\a'	bell (beep once)

Character constants

- **Escape character**

- **Character constants, like 'A', '\n', '0', '\0', use one byte and are enclosed in single quotes.**
- **'\ ' is an escape character, enabling special representations:**
 - (1) **'\n' for newline**
 - (2) **'\r' for carriage return**
 - (3) **'\t' for tab**
 - (4) **'\0' for string terminator (ASCII 0)**
 - (5) **'\'' for single quote**
 - (6) **'\ddd' for octal ASCII (max '\377')**
 - (7) **'\xdd' for hex ASCII (max '\xFF')**
 - (8) **'\a' for bell (same as '\007')**

Chapter Syllabus

- **2.1 Representation of Data in Computers**
 - 2.1.1 Computer Number Systems
 - 2.1.2 Representation of Numbers in Computers
- **2.2 Constants and Variables**
- **2.3 Basic Data Type Constants**
 - 2.3.1 Integer Constants
 - 2.3.2 Floating-point Constants
 - 2.3.3 Character Constants
- **2.4 Definition of Basic Data Type Variables**
 - 2.4.1 Definition of Integer Variables
 - 2.4.2 Definition of Floating-point Variables
 - 2.4.3 Definition of Character Constants

Definition of integer variables

- The format is as follows:

Type **Variable_List**

Type: unsigned short [int], short [int], unsigned [int], int, unsigned long [int], long [int], unsigned long long, long long, where [int] means “int” can be omitted

- Instructions:
 - **Variables of the same type** can be defined in one or more statements, separated by ",".
 - **Defined variables only allocate storage space** for the specified data type, with initial values being random until assigned.
 - C allows **simultaneous variable definition and initialization**.
 - When initializing long integers with basic integer constants, C automatically **converts the data type to match**.
 - Integer variables have a limited storage capacity, thus can only **store integers within a specific range**.
 - **"Define first, then use", "Write once, read many times"**.

Definition of integer variables

- **Example 2-16: Read the following C programs:**

```
#include <stdio.h>
main()
{
    long x, y, z;
    long long k;
    x = -0xffffL; y = -0xffL; z = -(long)0xffffffffL;
    printf("x=%6ld y=%6ld z=%ld\n", x, y, z);
    x = -0xffff; y = -0xff; k = -(long long)0xffffffff;
    printf("x=%6ld y=%6ld k=%lld\n", x, y, k);
}
```

- After running the program, the output result is

x=-65535 y= -255 z= 1

x=-65535 y= -255 z= -4294967295

- **Note:** (1) When defining data types, avoid the data truncation issue. (2) The data is stored using two's complement format. (3) Notice that the sign bit also occupies a bit of storage space.

Definition of integer variables

- **Example 2-17**

```
#include <stdio.h>
int main() {
    short x;
    unsigned short y;
    long z;
    x = 65535;
    y = 65535;
    z = 65535;
    printf("x=%d\n", x);
    printf("y=%u\n", y);
    printf("z=%ld\n", z);
    return 0;
}
```

- **Output Result:**

x=-1
y=65535
z=65535

- **%d, %u, %ld** are the format specifier for basic integer output, unsigned basic integer output, long integer output, respectively
- Variable x, y, z are represented in binary on the computer as follows:

Variable x	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
Variable y	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
Variable z	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Definition of integer variables

- **Example 2-17B**

```
#include <stdio.h>
int main() {
    short x;
    unsigned short y;
    long z;
    x = 75535;
    y = 75535;
    z = 75535;
    printf("x=%d\n", x);
    printf("y=%u\n", y);
    printf("z=%ld\n", z);
    return 0;
}
```

- **Output Result:**

x=9999
y=9999
z=75535

- **Illustration:** For short integer variables x and y, **two bytes** are not enough to store the binary value of 75535, and **can only store the binary value of the last 16 bits**, which corresponds to the decimal value of 9999.

Definition of real variables

Single precision

- Range : 6-7 significant digits
- Definition: float variable_list
- Allocation: 4B storage space

Double precision

- Range: 15-16 significant digits
- Definition: double variable_list
- Allocation: 8B storage space

- A real type declaration can define multiple variables of the same type, with each variable separated by a comma “,”.
- These variables can also be defined across multiple declarations.

Definition of real variables

Example:

```
#include <stdio.h>
int main() {
    float a = 1.0f, b = 2.5f, c;
    double d = 1.0, e, f;
    c = 755.35f;
    e = 5.35e-5;
    f = 1e2;
    printf("a=%f\n", a);
    printf("b=%f\n", b);
    printf("c=%f\n", c);
    printf("d=%f\n", d);
    printf("e=%f\n", e);
    printf("f=%f\n", f);
    return 0;
}
```

Output results:

```
a=1.000000
b=2.500000
c=755.349976
d=1.000000
e=0.000053
f=100.000000
```

Press any key to continue...

- %f outputs floats with 6 decimals, padding with zeros if needed.

Definition of character variables

- Character variables are used to **store character constants**.
- The way to define character variables is as follows: **char/bool variable_list;**
- Use **unsigned char for Chinese info** in GBK (2 chars per Chinese character).
- Chars can be used as integers, but **beware of overflow due to their limited range**.
- **Example 2-18: Output of Character Data and Integer Data.**

```
#include <stdio.h>
#include <stdbool.h>
int main() {
    int x = 65;
    char y = 'B';
    bool z = true;
    bool m = false;
    bool n = "a";
    printf("x=%c\t", x);
    printf("y=%c\t", y);
    printf("y=%d\t", y);
    printf("z=%d\t", z);
    printf("m=%d\t", m);
    printf("n=%d\t", n);
    return 0;
}
```

- The running result is:
x=A y=B y=66 z=1 m=0 n=1
- **Illustration:** The C compiler allocates one byte for char variables, **storing the ASCII code of the character**. So, storing a character constant actually means storing its ASCII code.

Definition of character variables

- Character type (ASCII code)

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Definition of character variables

- Character type (ASCII code)

Address	Data In Memory								ASCII	int
'ox10000000'	0	1	0	0	1	0	0	1	'I'	73
'ox10000001'	0	0	1	0	0	0	0	0	' '	32
'ox10000002'	0	1	0	0	1	1	0	0	'L'	76
'ox10000003'	0	1	1	0	1	1	1	1	'o'	111
'ox10000004'	0	1	1	1	0	1	1	0	'v'	118
'ox10000005'	0	1	1	0	0	1	0	1	'e'	101
'ox10000006'	0	0	1	0	0	0	0	0	' '	32
'ox10000007'	0	1	0	1	0	1	0	1	'U'	85



Definition of character variables

- **Example:**

```
#include <stdio.h>
int main() {
    int x;
    x = 1348; // 0x544
    printf("x=%c\n", x);
    return 0;
}
```

- The output result after running this program is:

x=D /* ASCII Value: 0x44 */

- **chars and ints are interchangeable**, allowing mixed operations and assignments.
- An int takes 4B, while a char takes 1B. When outputting an int as a char, **only the low byte is used for ASCII output.**
- **Chars can only hold 0-255.**

Definition of character variables

- **Example 2-19: Conversion between uppercase and lowercase English letters.**

```
#include <stdio.h>
int main() {
    char x, y, c1, c2;
    x = 'A';
    y = 'B';
    c1 = x + 32;
    c2 = y + 32;
    printf("x=%c, y=%c\n", x, y);
    printf("c1=%c, c2=%c\n", c1, c2);
    return 0;
}
```

- The output result after running this program is:
x=A, y=B
c1=a, c2=b
- Each lowercase letter's ASCII code is 32 more than its uppercase version.
- $c1 = x + 32$; can be written as: $c1 = x - 'A' + 'a'$;
 $c2 = y + 32$; can be rewritten as: $c2 = y - 'A' + 'a'$;
Character case conversion is achieved through calculation of relative positions.

Definition of constant variables

- Constant variables, also referred to as **read-only variables**, serve to safeguard **data security**.
- They **prevent accidental data modifications**.
- The declaration format is: "**const type variable_name**".
- Example: **const int a=3; const float b=3.14*2.**

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char a = 'a';
```

```
    const char b = 'b';
```

```
    a = a - 32;
```

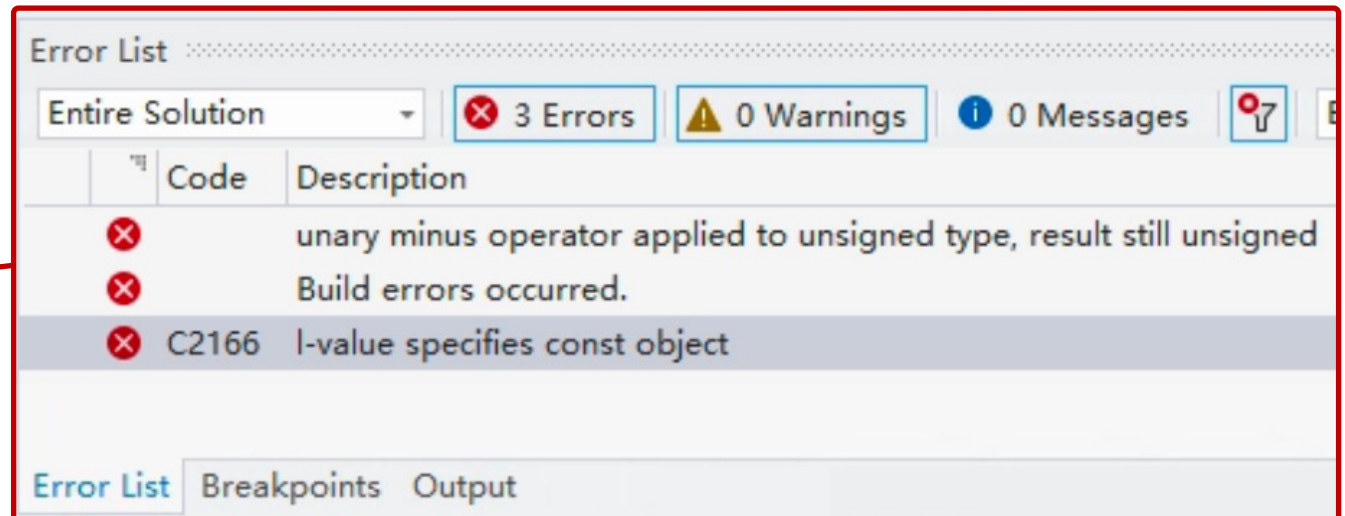
```
    b = a - 32;
```

```
    printf("a=%c\n", a);
```

```
    printf("b=%c\n", b);
```

```
    return 0;
```

```
}
```



Summarize the basic types in C

- Including **arithmetic types, enumerated types, and the void type**. Among them, arithmetic types include character types, integer types, and floating-point types. (Enumerated types and the void type will be discussed later.)

Data Type	Keyword	Length Description	Value Range	Space
Integer data	(unsigned) short int	2B (un)signed integer	$[0, 2^{16}-1]$ or $[-2^{15}, 2^{15}-1]$	2G
	(unsigned) int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$	4G
	(unsigned) long int	4B (un)signed integer	$[0, 2^{32}-1]$ or $[-2^{31}, 2^{31}-1]$	4G
	(unsigned) long long	8B (un)signed long integer	$[0, 2^{64}-1]$ or $[-2^{63}, 2^{63}-1]$	9E
Floating-point data or real number data	float	4B floating-point number	$\sim(-10^{38} \sim 10^{38})$	
	double	8B floating-point number	$\sim(-10^{308} \sim 10^{308})$	
	long double	8B floating-point number	$\sim(-10^{308} \sim 10^{308})$	
Character data	char	1B signed character	$[-2^7, 2^7-1]$	
	unsigned char	1B unsigned character	$[0, 2^8-1]$	
	bool	1B unsigned character	$\{0, 1\}$	

**How to set the Release mode
for the VS2022 compilation system?**

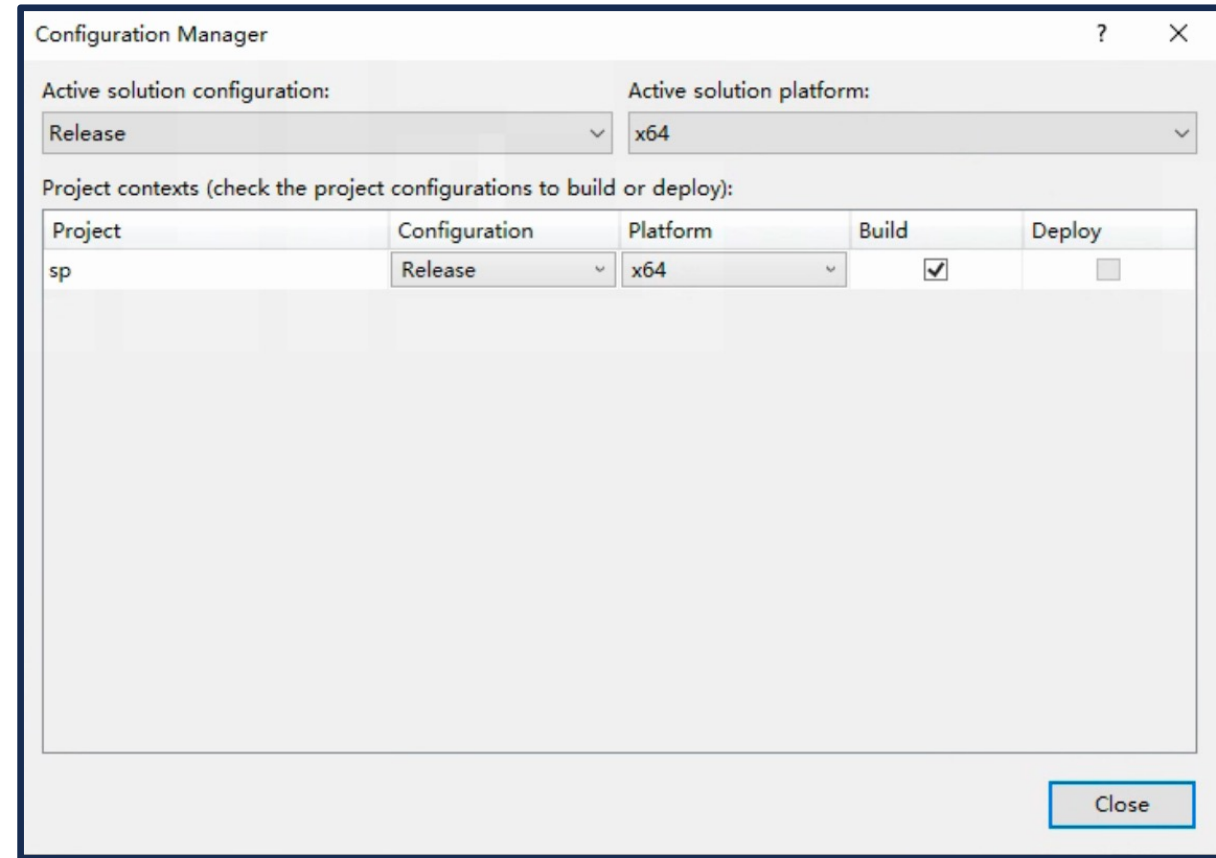
VS2022 compilation system

- The VS2022 compilation system has **two compilation modes: Debug mode and Release mode.**
- **Debug mode:** When you create a project for compilation and linking, the **compilation system automatically defaults to Debug mode.** You can see “Configuration: Debug x64” in the compilation information.
- **Efficiency of Debug mode:** **Debug mode aids in error tracing,** but the resulting executable file obtained through compilation and linking is **larger and slower than in Release mode** due to added **debugging and tracing information.**
- **Release mode:** After **confirming that there are no errors in your program in Debug mode,** you can **switch to Release mode** and recompile and link to **generate the final executable program.**

VS2022 compilation system

- **Active release mode:**

1. Select the “Configuration Manager” item in the “Build” menu.
2. Click the "Active Solution Configuration" dropdown, choose "Release", and close. This activates Release mode.
3. After rebuilding solution, you can see “Configuration: Release x64” in the compilation information.

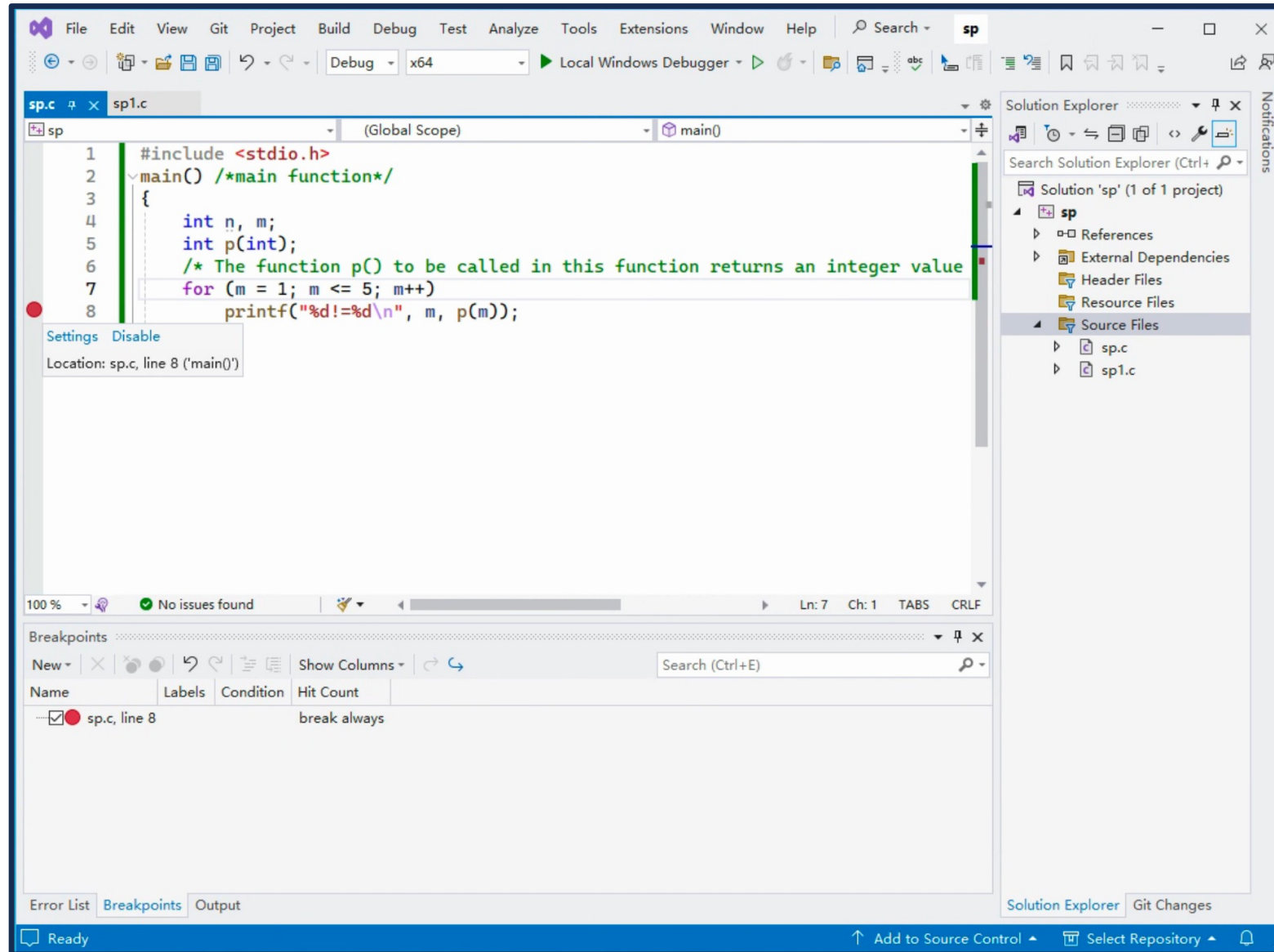


How to track and debug logical errors in programs

VS2022 debug system

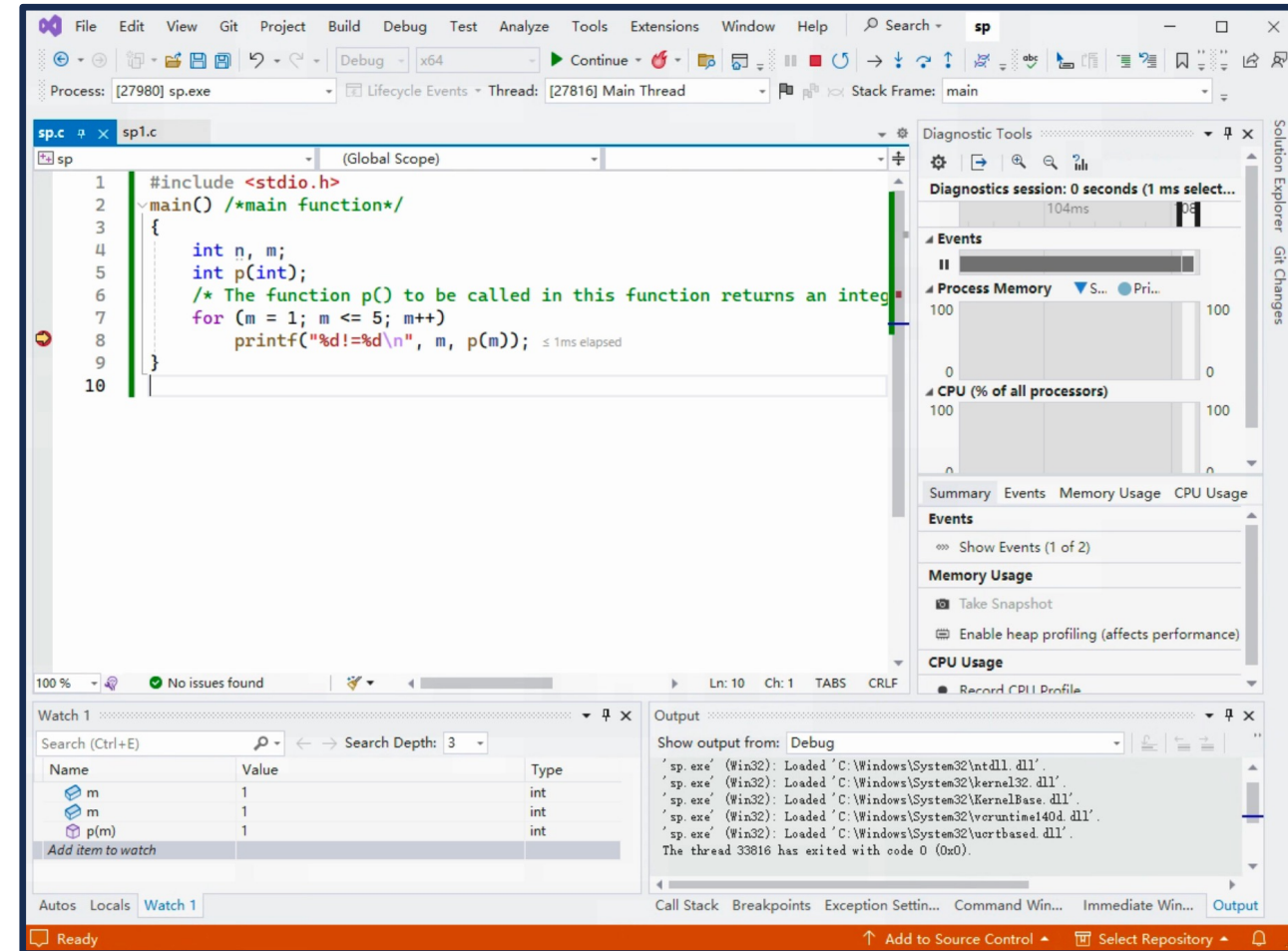
- In Debug mode, at potential error lines, **use F9, click the margin, or choose "Toggle Breakpoint" in the "Debug" menu to set/remove a breakpoint.** Hovering over the breakpoint shows a "Breakpoint" prompt.
- Lines with breakpoints display a large dot. **They pause the program at that line, letting you check variable values.**
- **Note: You can set multiple breakpoints in a program.**

VS2022 debug system



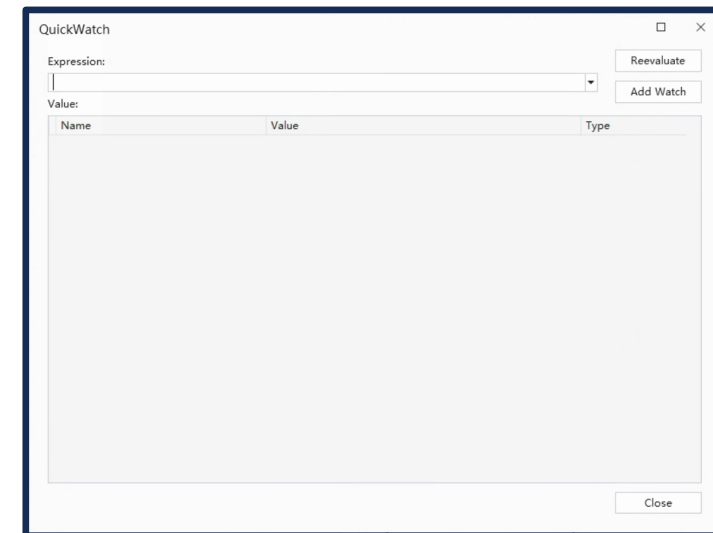
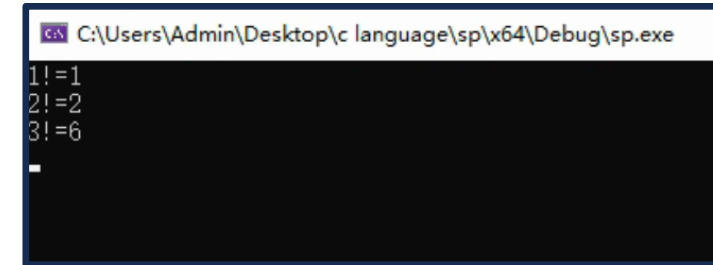
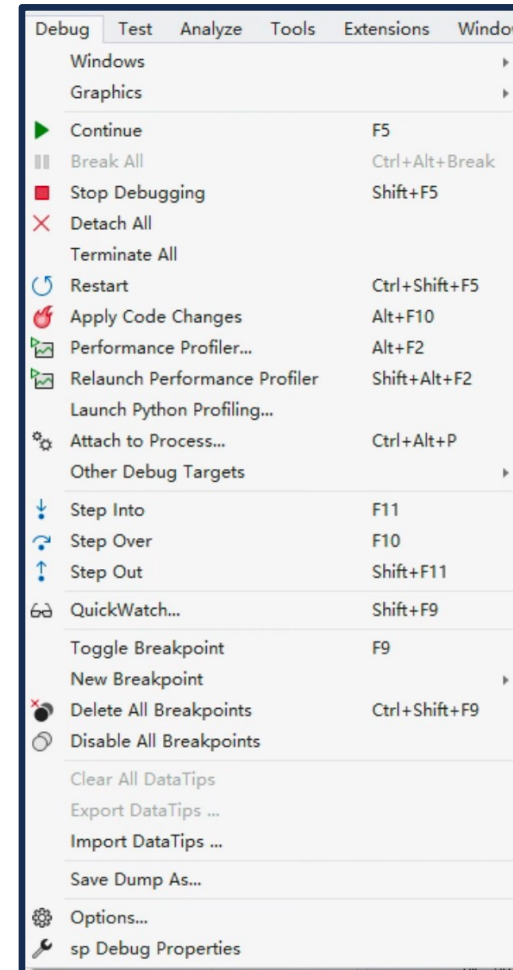
VS2022 debug system

- Select the **'Debug'** menu and you will see the **various functions**
- After setting breakpoints, start debugging with **"Start Debugging"**, **F5**, or click the icon. The program halts at breakpoints, showing current variable values.



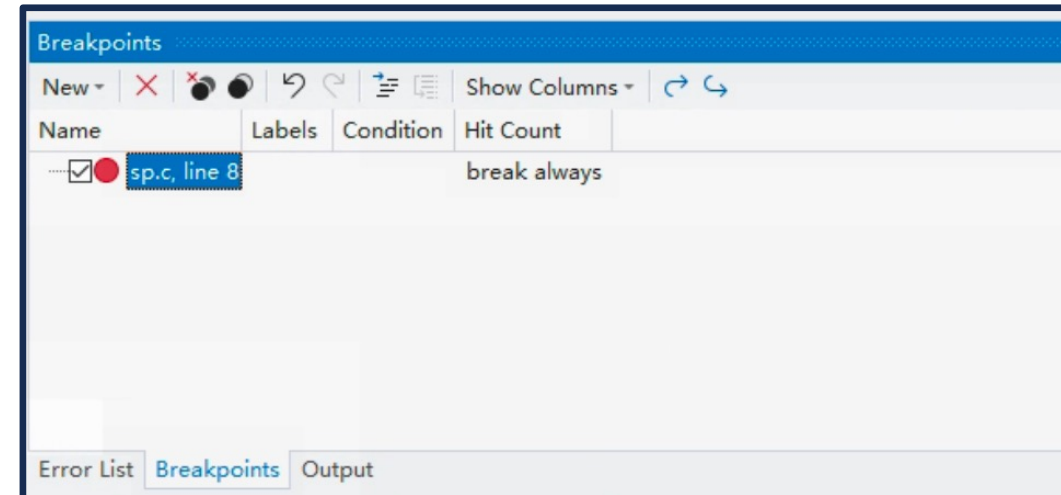
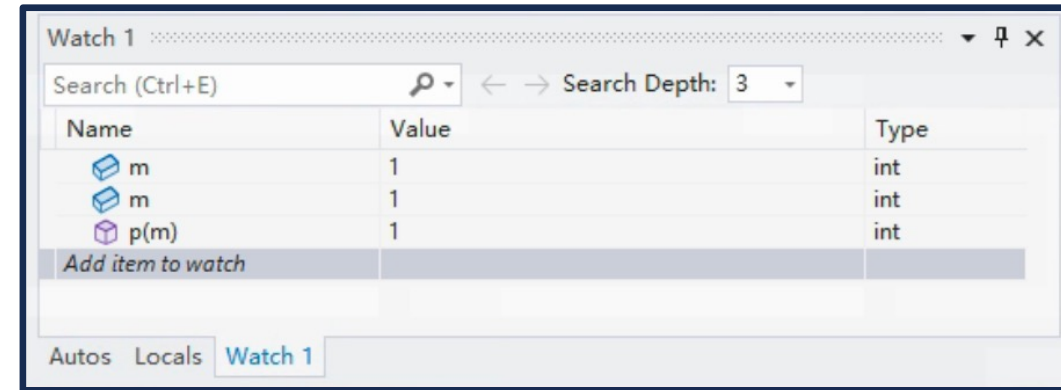
VS2022 debug system

- Checking the **“Debug” menu** again reveals useful functions, e.g., **“Continue” function**.
- Use **F10 (step over)** and **F11 (step into)** to step through the program, observe variable changes, and pinpoint errors.
- Shortcuts like **Shift-F11** and **Ctrl-F10 (run to breakpoint)** offer slight variations in stepping through code—explore them!
- Use **Shift-F9** for a quick watch window, monitoring designated variables or expressions (see figure).



VS2022 debug system

- In the “Autos” window, **enter the variable to track in the “Name” column. Press Enter to see its current value.**
- Use **“Stop Debugging” in the “Debug” menu or Shift-F5 to end debugging. Modify code or use Ctrl-Shift-F5 to restart.**
- Move cursor, use **F9 to remove breakpoints, recompile, and relink. If no errors, try Release mode.**
- **Note: Correct Debug mode doesn't guarantee correctness in Release mode.**



Explore the Leetcode programming platform

- **Leetcode programming platform:** <https://leetcode.com/>
- **Chinese Leetcode platform:** <https://leetcode.cn/>
- **Support of Leetcode platform for learning C language programming**
 - Question bank: practice of algorithm and data structure
 - Online programming environment
 - Community interaction and discussion
 - Recommendations for learning resources
 - Real-time rankings and competitions

Homework

- **2nd assignment: Book p.43, Exercises 3, 4**

- **2nd computer-based experiments**

- (1) Write a program to print and view the double-precision floating-point values of 97.6875 and -97.6875. Display their IEEE 754 storage format in hexadecimal by bytes to verify if the results are consistent with those given in the textbook. Also, print and view the single-precision floating-point values (float) of 97.6875 and -97.6875 in the computer, showing their IEEE 754 storage format in hexadecimal by bytes.**
- (2) Write a program to print the hexadecimal representation by bytes of the storage of 100 and -100 in the computer for double, float, long, int, short, and char data types respectively. Analyze why the results are different for each data type.**

Homework

- **Attached:** An example program that prints the hexadecimal representation of the internal values of double and float type variables:

```
#include <stdio.h>
int main() {
    double x = 97.6875;
    float y = 97.6875;
    unsigned char *p;
    p = (unsigned char *)&x;
    printf("%02X %02X %02X %02X %02X %02X %02X %02X\n",
        *p, *(p+1), *(p+2), *(p+3), *(p+4), *(p+5), *(p+6), *(p+7));
    p = (unsigned char *)&y;
    printf("%02X %02X %02X %02X\n", *p, *(p+1), *(p+2), *(p+3));
}
```

- The program sequentially prints the hexadecimal value of each byte pointed to by the pointer.